DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

UNIVERSITÀ
DEGLI STUDI
DI PADOVA




# LAB 2: Road Lines and Street Signs Detection with the Hough Transform

Computer Vision 2024-25

M. Caligiuri, F. Lincetto, F. Barbato, P. Zanuttigh

# CV 2024-25: HW / Projects

Intro to Python/ OpenCV — No HW

1. Histogram proc. and Filtering
2. **Road Line Detection**

- Select 1 among LAB1 and LAB2
- First «simple» HW: 2 points
- Deliver the jupyter notebook file with code and comments
- On/off mark
- Deadline 29/11

3. Feature Descriptors
4. Deep Learning for CV

- Select 1 among 3 and 4
- Second «advanced» HW: 5 points
- Deliver code, report and results
- Mark based on solution quality

Final Exam (written in classroom): ~25 points

Final mark: **2** + 5 + 25 = max 32 points

DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

# LAB2: Find Road Lines and Street Signs in an Image







Detect the street lines and the circular road signs:

1. Find the edges (e.g., with Canny)
2. Find the street lines with the Hough Transform, first using OpenCV and then manually implementing the Hough Transform
   - Support functions to visualize the output are provided
3. Find the circles corresponding to road signs
   - Using the Hough Transform for circles (in this case use only OpenCV implementation)

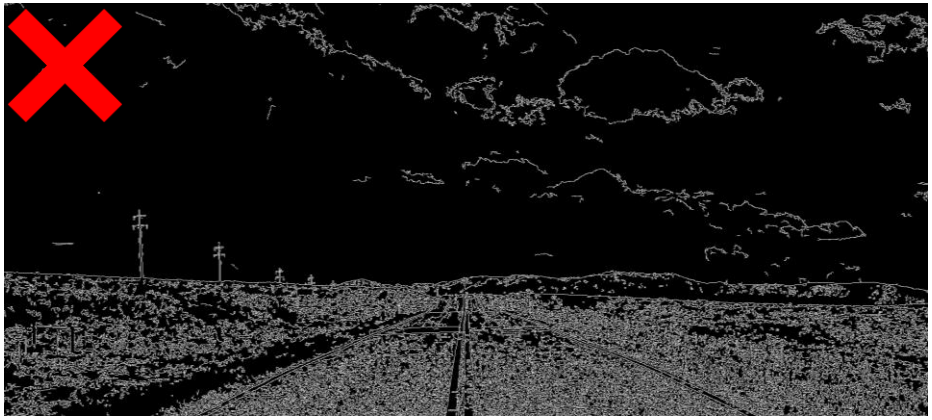
# Recall: Canny Edge Detector

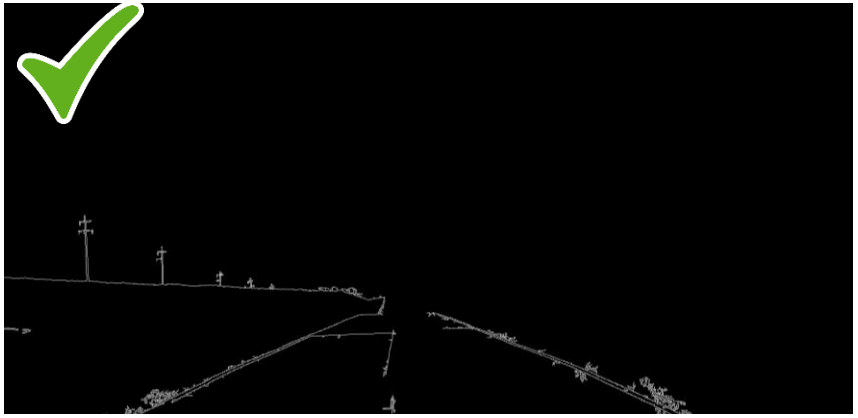




1. Smoothing with a Gaussian filter (param: *std. dev σ*)
2. Compute gradient (module and direction)
3. Quantize the gradient angles
4. Non-maxima suppression
5. Thresholding with double threshold (params: *2 thresholds*)

# Canny: Parameters





The Canny algorithm has 3 parameters:

1. $\sigma$ : Gaussian smoothing, find only large structures or small details, controls robustness to noise
2. $T_l$: Low threshold, keep low for edge linking (recall points between $T_l$ and $T_h$ are selected only if connected to points in $T_h$ )
3. $T_h$ : High threshold, sets the number of edges found (increase to avoid false detections)

*Before searching for lines with Hough Tr. verify the edge detector output !*

# Canny in OpenCV

```
void cv2.Canny    (    input_image,
                       threshold1,
                       threshold2,
                       apertureSize,
                       L2gradient
                  )
```

The Canny algorithm is implemented in OpenCV

- *threshold1*, *threshold2* ↔ $T_l$, $T_h$ *(automatically gets the largest)*
- *apertureSize* ↔ σ (smoothing implemented inside the gradient extraction, *default=3*)
- *L2gradient* (*default=false*): use $\sqrt{\left(\frac{\partial f_s}{\partial x}\right)^2 + \left(\frac{\partial f_s}{\partial y}\right)^2}$ *or* $\left|\frac{\partial f_s}{\partial x}\right| + \left|\frac{\partial f_s}{\partial y}\right|$

# Hough Transform Algorithm

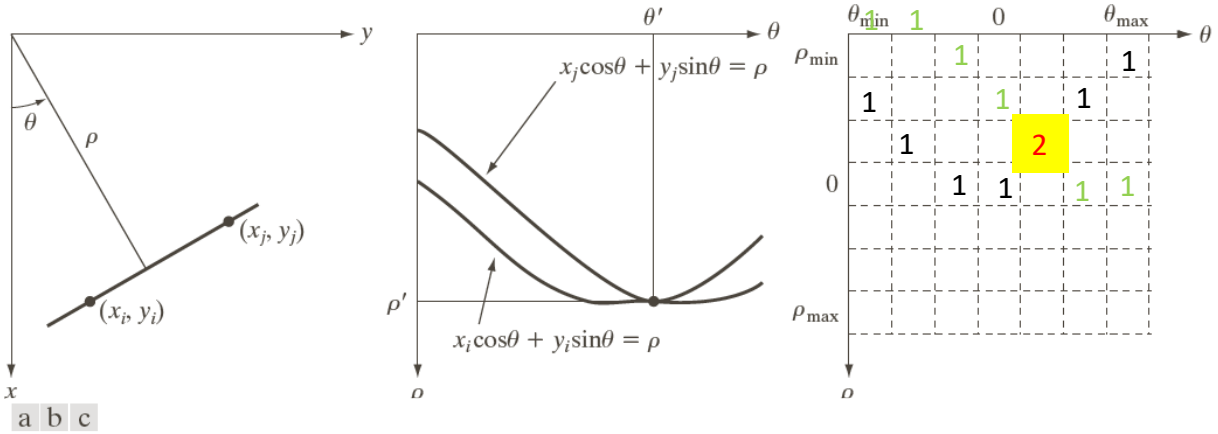
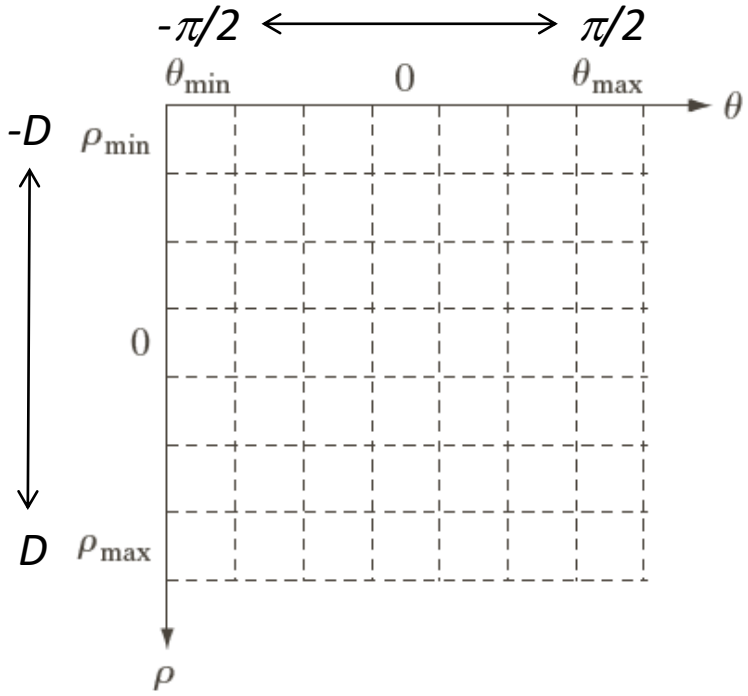



**FIGURE 10.32** (a) $(\rho, \theta)$ parameterization of line in the $xy$-plane. (b) Sinusoidal curves in the $\rho\theta$-plane; the point of intersection $(\rho', \theta')$ corresponds to the line passing through points $(x_i, y_i)$ and $(x_j, y_j)$ in the $xy$-plane. (c) Division of the $\rho\theta$-plane into accumulator cells.

Steps to implement HT

1. Compute edge detection (e.g., with `cv2.Canny`) and get edge points
2. The polar parameter space is quantized in cells, there is a counter for each cell (the function to convert to polar coordinates is provided in the notebook, initialize a 2D matrix)
3. For each edge pixel:
   A. Let θ vary on the quantized interval $(-\frac{\pi}{2}, \frac{\pi}{2})$ in uniform steps and compute the corresponding ρ values (recall: $\rho = xcos\theta + ysin\theta$ )
   B. For each crossed cell increment by 1 the counter in the matrix
4. The counter for each cell contains the number of pixels collinear on that line
   - Use a threshold on the counter values to get the lines

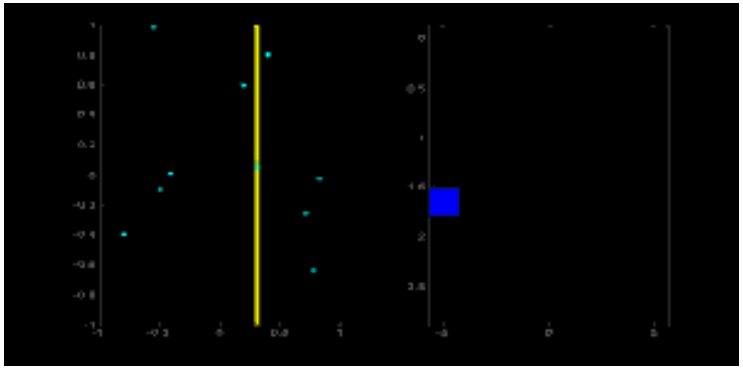
# Hough Transform: Accumulation Cells

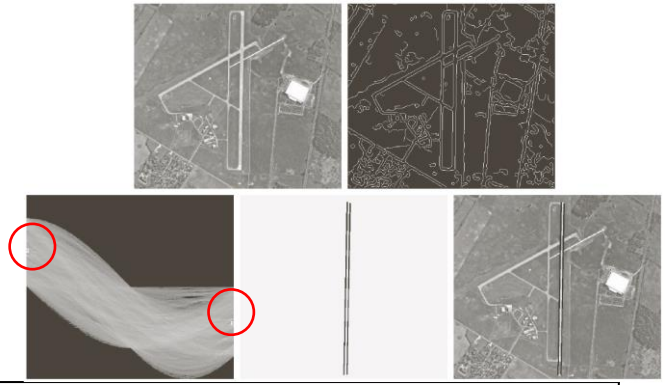



| Few Large Cells | Many Small Cells |
|---|---|
| Accepts pixels not perfectly aligned | Requires precise alignment |
| Stable w.r.t. noise | Sensitive to noise |
| Poor lines localization | Accurate lines localization |
| Fast | Slow |

- The parameter space is quantized along ρ and θ
  - ρ and θ: distance from the origin and orientation
- The cell subdivision allows to handle points not perfectly aligned
- *Cell size*: quantization of the line localization params

# OpenCV: HoughLines





```
lines = cv2.HoughLines   ( image,
                           rho,              Cell size (ρ-dim) in [pxl]
                           theta,            Cell size (θ-dim) in [rad]
                           threshold,        Min # of points for line
                           lines,            Array of 2-elements
                                             vectors (ρ,θ)
                           srn = 0,          For multi-resolution
                           stn = 0,          For multi-resolution
                           min_theta = 0,    Angle min (0=vertical)
                           max_theta = CV_PI Angle max
                         )
```

*Note*: «0» is vertical direction and range cannot cross 0: need 2 searches for close to vertical lines

DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

# Get Line Positions and Draw Lines

```
# Funtion to add lines to an image
def draw_lines(img: np.ndarray, lines: np.ndarray, color: tp.List[int] = [0, 0, 255],
thickness: int = 1) -> tp.Tuple[np.ndarray]:
    new_image = np.copy(img)
    empty_image = np.zeros(img.shape[:2])

    if len(lines.shape) == 1:
        lines = lines[None, ...]

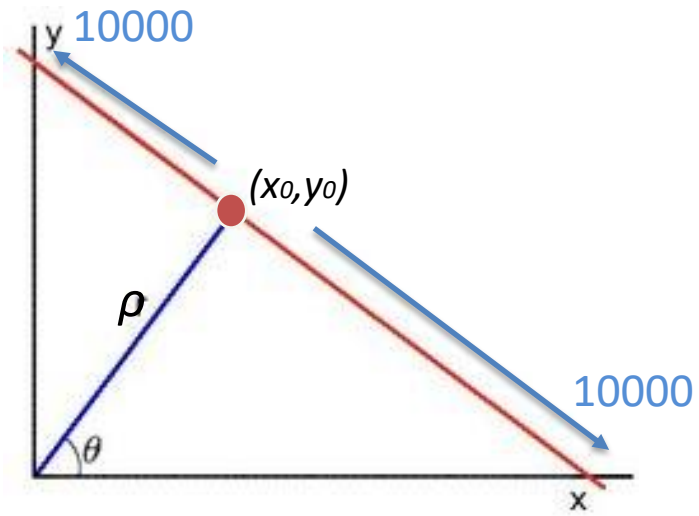
    # Draw found lines
    for rho, theta in lines:
        cx, direction = polar2cartesian(rho, theta)
        pt1 = np.round(cx + 10000*direction).astype(int)
        pt2 = np.round(cx - 10000*direction).astype(int)
        empty_image = cv.line(img=empty_image,pt1=pt1, pt2=pt2, color=255,
thickness=thickness)

    # Keep lower part of each line until intersection
    mask_lines = empty_image != 0
    max_line = 0
    valid = flag = False # check that we found 2 lines at least once before
    for i, line in enumerate(mask_lines): # iterate each line and search for a batch of
contiguous idxs
        indices = np.argwhere(line)
        if len(indices) > 1:
            flag = True
            for ii in range(len(indices)-1):
                flag = flag and indices[ii+1] == indices[ii] + 1
                if not flag:
                    valid = True
                    break
                if valid:
                    max_line = i
        elif len(indices) == 1 and valid:
            max_line = i
        if flag and valid:
            break

    mask_boundaries = np.zeros_like(empty_image)
    mask_boundaries[max_line:] = 1
    mask = (mask_lines * mask_boundaries).astype(bool)

    new_image[mask_lines] = np.array(color)

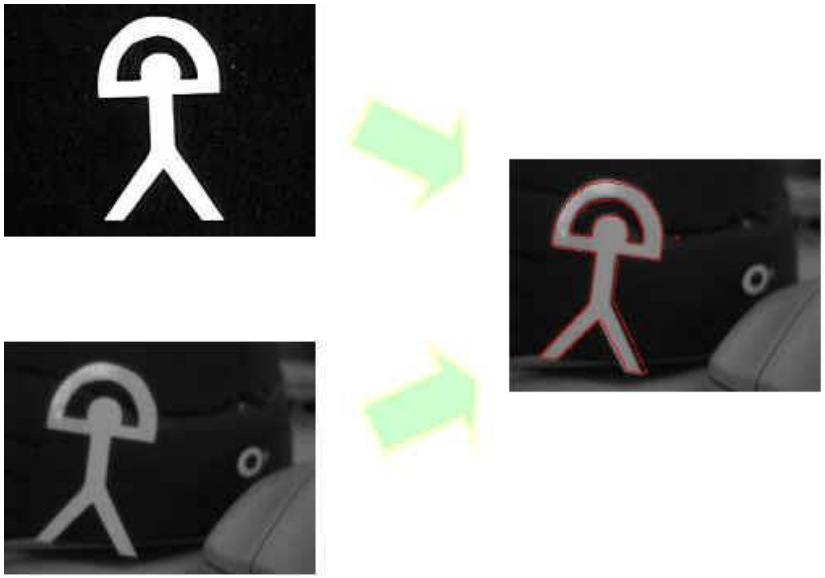
    return new_image, mask
```



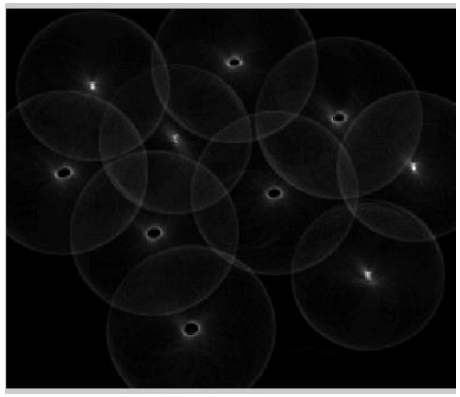

*Note:* OpenCV has 0° corresponding to vertical axis
The cv2_setup bool in the notebooks switch between this convention and the standard one with 0°=horizontal

*The function is provided in the notebook!*

# Hough Transform for Circles










$g(\mathbf{v},\mathbf{c})=0$ → $(x - c_1)^2 + (y - c_2)^2 = c_3^2$

*variables* — $\mathbf{v}$

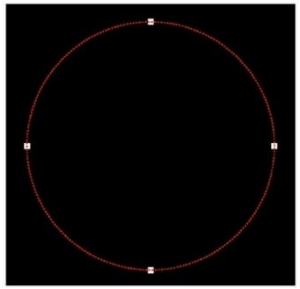
*parameters* — $\mathbf{c}$

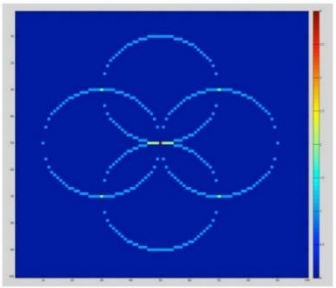
Center: $(c_1, c_2)$
Radius: $c_3$



Dimensionality of vector **c** (parameter space) is 3

# cv2.HoughCircles





*cv2.circle plots a circle over the image*

```
circles=
cv2.HoughCircles      (     image,          Grayscale input image (edge detection performed inside the function)
                            circles,        List of found circles: each element is a 3-element floating-point vector (x, y, radius)
                            method,         HOUGH_GRADIENT (only available option)
                            dp,             Image resolution / accumulator resolution (ratio)
                            minDist,        Minimum distance between two circles
                            param1 = 100,   Th of Canny (Tl = Th / 2)
                            param2 = 100,   Threshold for accumulator count to detect a circle
                            minRadius = 0,  Minimum radius
                            maxRadius = 0   Maximum radius (set small to avoid large circles)
                      )
```

*Note*: the threshold on # of points favours large circles even if interrupted, use max radius to prevent this